

Linguagem C

Prof. Isaac Benjamim Benchimol
ibench@ifam.edu.br

Funções

Funções

- É uma sub-rotina que realiza uma tarefa específica.
- Um programa em C é uma coleção de funções.
- Forma geral de uma função:

```
tipo_retorno nome(lista_parâmetros)
{
    // corpo da função
}
```

`tipo_retorno` especifica o tipo de dado retornado pela função. Pode ser qualquer tipo válido exceto um *array*.

`nome` especifica o nome da função.

`lista_parâmetros` é uma seqüência de pares de tipo e identificador separados por vírgulas. São as variáveis que recebem o valor dos argumentos passados para a função quando chamada.

Função sem parâmetro

```
// Este programa contém duas funções: main() e
    minha_func()
void minha_func();    // protótipo de minha_func()
                        // Não há parâmetros

int main(){
    printf("Em main() \n");
    minha_func();      // chame minha_func()
    printf("De volta a main()");
}

// Esta é a definição da função.
void minha_func(){
    printf("De dentro de minha_func()\n");
}
```

Em main()
De dentro de minha_func()
De volta a main()

Função com parâmetro

```
void box(int a, int b, int c);    // protótipo
de box() com 3 parâmetros inteiros a, b e c
int main(){
    box(7, 20, 4); // passa argumentos para box
    box(28, 30, 25);
    box(3, 5, 4);
}
// Computa o volume da caixa.
void box(int a, int b, int c){
    printf("Volume da caixa = %d\n", a*b*c);
} // Os parâmetros recebem os valores dos
    argumentos passados para box().
```

Regras de Escopo

- Existem três tipos de variáveis: variáveis locais, parâmetros formais e variáveis globais.
- As regras de escopo regem a visibilidade e o tempo de vida de cada um desses tipos.

Variáveis Locais

- Declaradas dentro de uma função ou dentro de um bloco de código.
- Conhecidas apenas dentro do bloco no qual estão declaradas.
- São criadas na entrada do bloco e destruídas na saída.

Variáveis Locais

```
void func1() {  
    int x;          //x é local para func1()  
    x=10;  
    printf("x = %d",x);  
}  
  
void func2(){  
    int x=9; //x é local para func2().  
    if (x==9){  
        int y=20; //y é local ao bloco if  
        printf("x + y = %d", x+y);  
    }  
    y=100; //erro! Y não é conhecido aqui.  
}
```

Parâmetros Formais

- **Parâmetros Formais**
 - São as variáveis que recebem os argumentos de uma função.
 - Dentro de uma função, elas se comportam como qualquer outra variável local.
 - Os parâmetros formais declarados devem ser do mesmo tipo dos argumentos usados para chamar a função.

Parâmetros Formais

```
#include <stdio.h>          /* declaração do arquivo de cabeçalho*/
int mul_var(int, int);      /* declaração do protótipo mul_var */
int mul_var(int x, int y) /* declaração da função mul_var */
{
    int z;
    z = x * y;
    return(z);
}

void main(void)              /* declaração da função main()
{
    int a, b, mul;
    printf("Digite 2 valores:\n");
    scanf("%d %d", &a, &b);
    mul = mul_var(a, b);    /* chamada da função mul_var*/

    printf("\n %d ", mul);
}
```

Parâmetros formais da função

Argumentos da função, devem ser do mesmo tipo do parâmetro formal

Variáveis Globais

- **Variáveis Globais**
 - Estas variáveis são declaradas fora das funções, ou seja, logo após as declarações `#include`.
 - São conhecidas por todo o programa e podem ser usadas por qualquer parte do código.
 - Variáveis globais retêm seus valores durante toda a execução do programa.

Variáveis Globais

```
#include <stdio.h>
void func1();
int count;          // count é global

int main() {
    int i;          // i é local
    for (i=0;i<10;i++) {
        count = i*2;
        func1();
    }
    return 0;
}

void func1() {
    printf("count: %d\n", count); //acessa count global
}
```

O retorno de valores

- Uma função pode retornar um valor para seu chamador.
- Para retornar um valor usamos `return valor`.
- O tipo do valor retornado deve ser compatível com o tipo de retorno declarado na função. Se não for, um erro em tempo de compilação ocorrerá.

Função com retorno

```
int box(int a, int b, int c);    // box() agora
    retorna um inteiro
int main(){
    int volume;
    volume = box(7, 20, 4); // o valor de
        retorno de box() é atribuído a volume.
    printf("O Volume da caixa e %d", volume);
}
// Computa o volume da caixa.
int box(int a, int b, int c){
    return a*b*c;
}    // box() retorna o volume da caixa para
    main()
```

O Volume da caixa e 560

Exercícios

1. Escreva uma função que receba três argumentos reais e retorne a média aritmética entre eles:
`float media(float x1, float x2, float x3)`
2. Escreva uma função que receba dois inteiros como argumentos e retorne o maior valor.
3. Escreva uma função que receba um valor inteiro e retorne o seu fatorial.
4. Escreva uma função que receba um valor inteiro n como argumento e retorne 1 se n for primo; caso contrário retorne 0.
5. Escreva a função `power()`, que retorna x elevado a potência de n , onde n pode ser qualquer inteiro:
`double power(double x, int p)`
Utilize o algoritmo que calcularia x^{20} multiplicando 1 por x 20 vezes.

Exercícios

6. O que será exibido pelo programa?

```
#include <stdio.h>

int confusion(int x, int y) {
    x = 2*x + y;
    return x;
}

int main(void) {
    int x = 2, y = 5;
    y = confusion(y, x);
    x = confusion(y, x);
    printf("%d %d\n", x, y);
    return 0;
}
```